

Section 8.1 - Introduction to ConvNets

November 28, 2025

1 Convolutional neural networks

1.1 Instantiate a ConvNet

- Functional API
- Conv2D(filters, kernel_size, activation). Default stride=1
- Number of filters: depth of the output feature map
- MaxPooling2D(pool_size, stride). Default stride = pool_size
- Flatten = vectorization operation -> A long row vector

```
[1]: from tensorflow import keras
from tensorflow.keras import layers
inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

1.2 Model summary

- Model architecture
- Understand the shape of each output tensor (Conv2D, MaxPooling2D)
- Number of weights per layer
- Output shape for 3-channel images: Conv2D filter spans all input channels

```
[2]: model.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 28, 28, 1)]	0
conv2d (Conv2D)	(None, 26, 26, 32)	320

max_pooling2d (MaxPooling2D	(None, 13, 13, 32)	0
)		
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling	(None, 5, 5, 64)	0
2D)		
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73856
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 10)	11530

=====

Total params: 104,202
Trainable params: 104,202
Non-trainable params: 0

1.3 Training on the MNIST images

- optimizer: rmsprop
- loss: sparse_categorical_crossentropy
- metrics: accuracy
- epochs: 5
- batch_size: 64

```
[3]: from tensorflow.keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype("float32") / 255

model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_images, train_labels, epochs=5, batch_size=64)
```

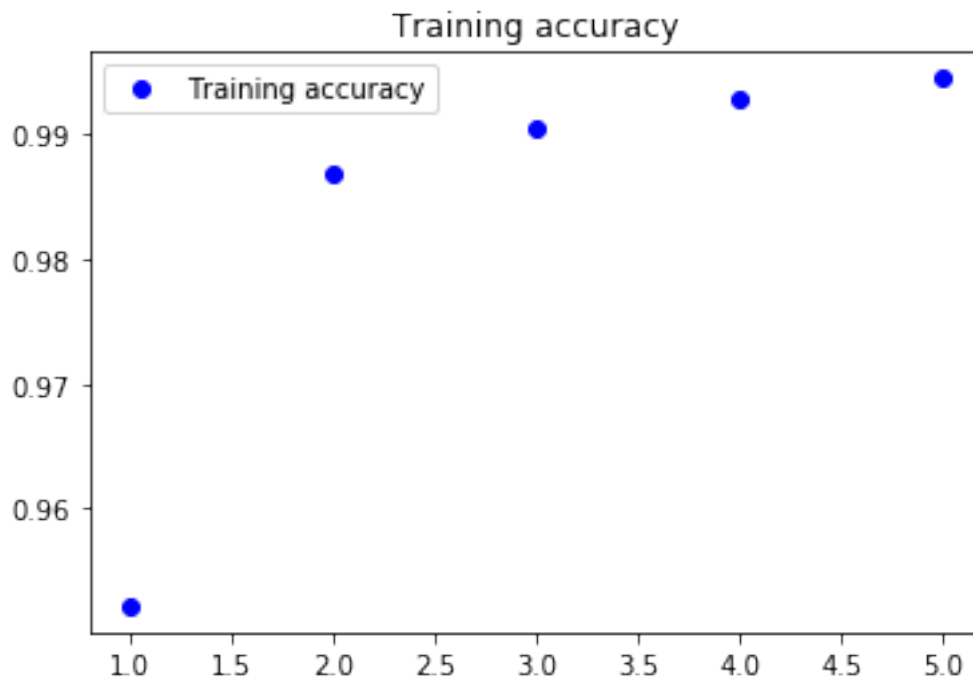
Epoch 1/5
938/938 [=====] - 3s 2ms/step - loss: 0.1553 -
accuracy: 0.9521
Epoch 2/5
938/938 [=====] - 2s 2ms/step - loss: 0.0445 -
accuracy: 0.9868

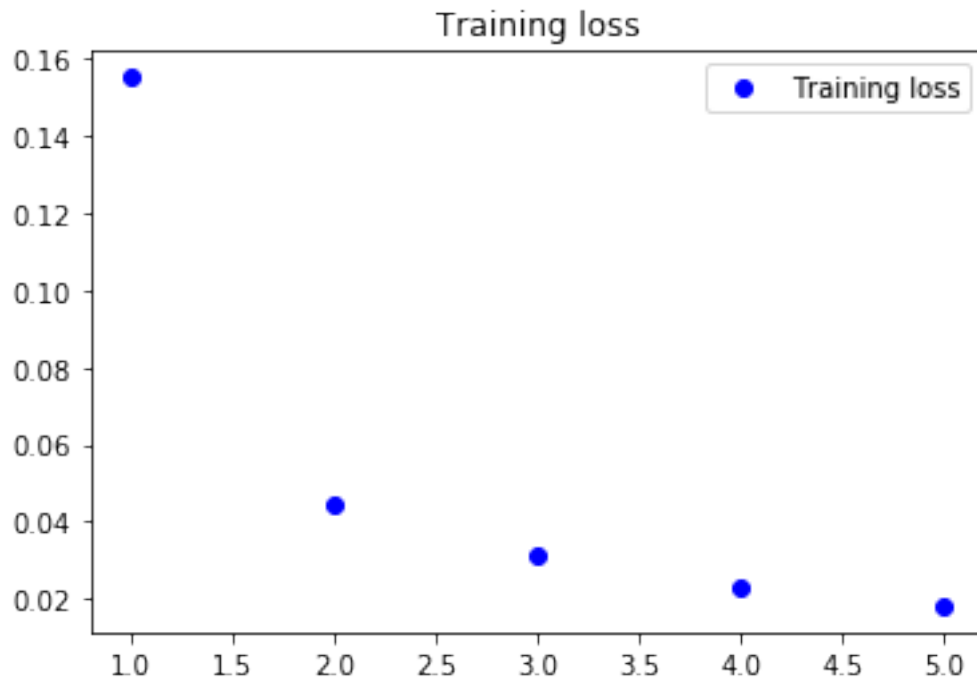
```
Epoch 3/5
938/938 [=====] - 2s 2ms/step - loss: 0.0315 -
accuracy: 0.9905
Epoch 4/5
938/938 [=====] - 2s 2ms/step - loss: 0.0228 -
accuracy: 0.9929
Epoch 5/5
938/938 [=====] - 2s 2ms/step - loss: 0.0181 -
accuracy: 0.9945
```

1.4 Plotting the training results

```
[4]: import matplotlib.pyplot as plt

acc = history.history["accuracy"]
loss = history.history["loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.title("Training accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.title("Training loss")
plt.legend()
plt.show()
```





1.5 Evaluation on test dataset

```
[5]: test_loss, test_acc = model.evaluate(test_images, test_labels)
     print(f"Test accuracy: {test_acc:.3f}")
```

```
313/313 [=====] - 1s 2ms/step - loss: 0.0293 -
accuracy: 0.9906
Test accuracy: 0.991
```